



Natural Language Processing: Seq2seq and Attention

HSE Faculty of Computer Science
Machine Learning and Data-Intensive Systems

Murat Khazhgeriev



Table of Content

- **Sequence to Sequence (seq2seq)**
- LSTM
- Attention
- Practical tips

Machine translation requires training parameters to provide results

Human Translation

$$y^* = \arg \max_y p(y|x)$$

The “probability” is intuitive and is given by a human translator’s expertise

Machine Translation

$$y' = \arg \max_y p(y|x, \theta)$$

model parameters

Questions we need to answer

- **modeling**

How does the model for $p(y|x, \theta)$ look like?

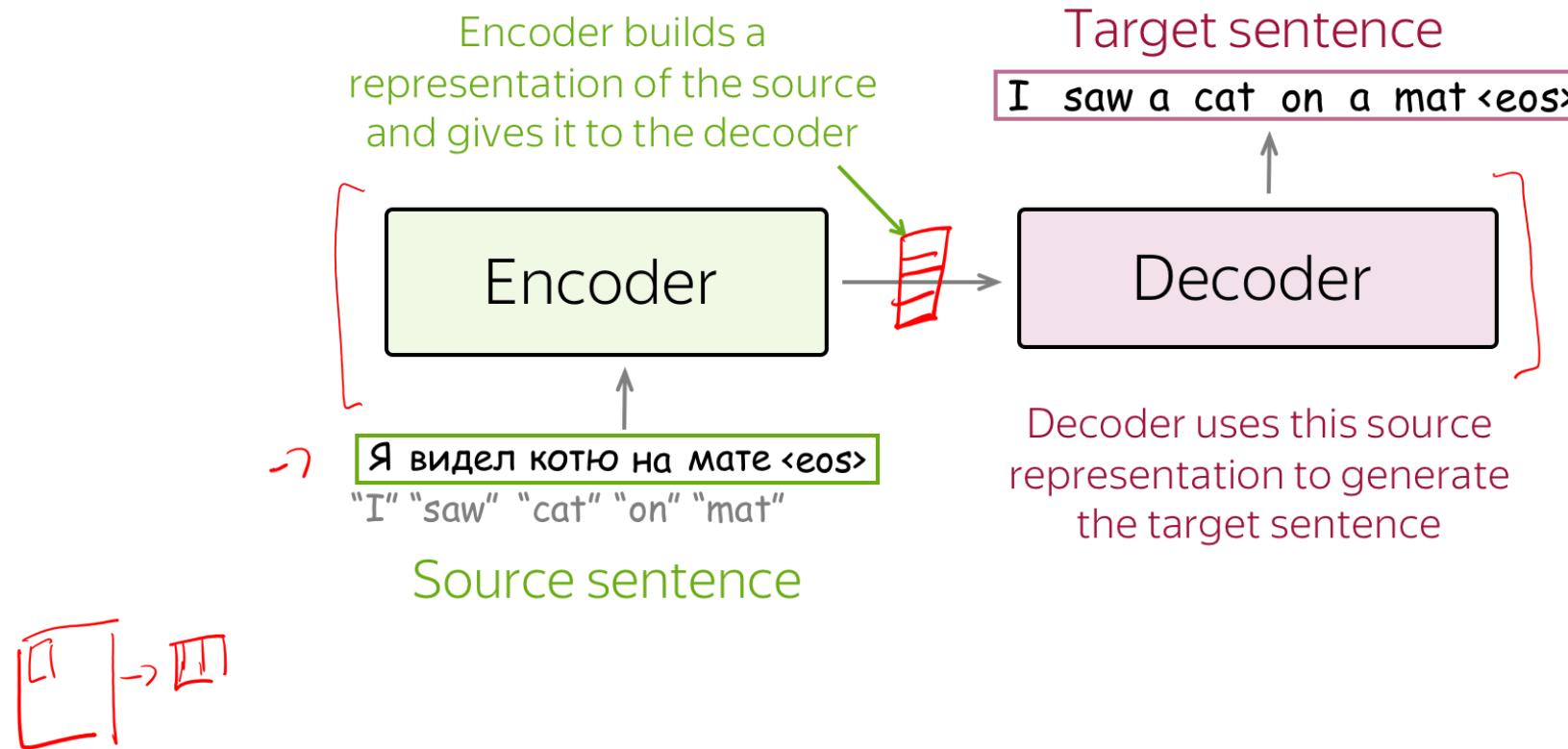
- **learning**

How to find θ ?

- **search**

How to find the argmax?

Encoder-decoder architecture maps data semantics



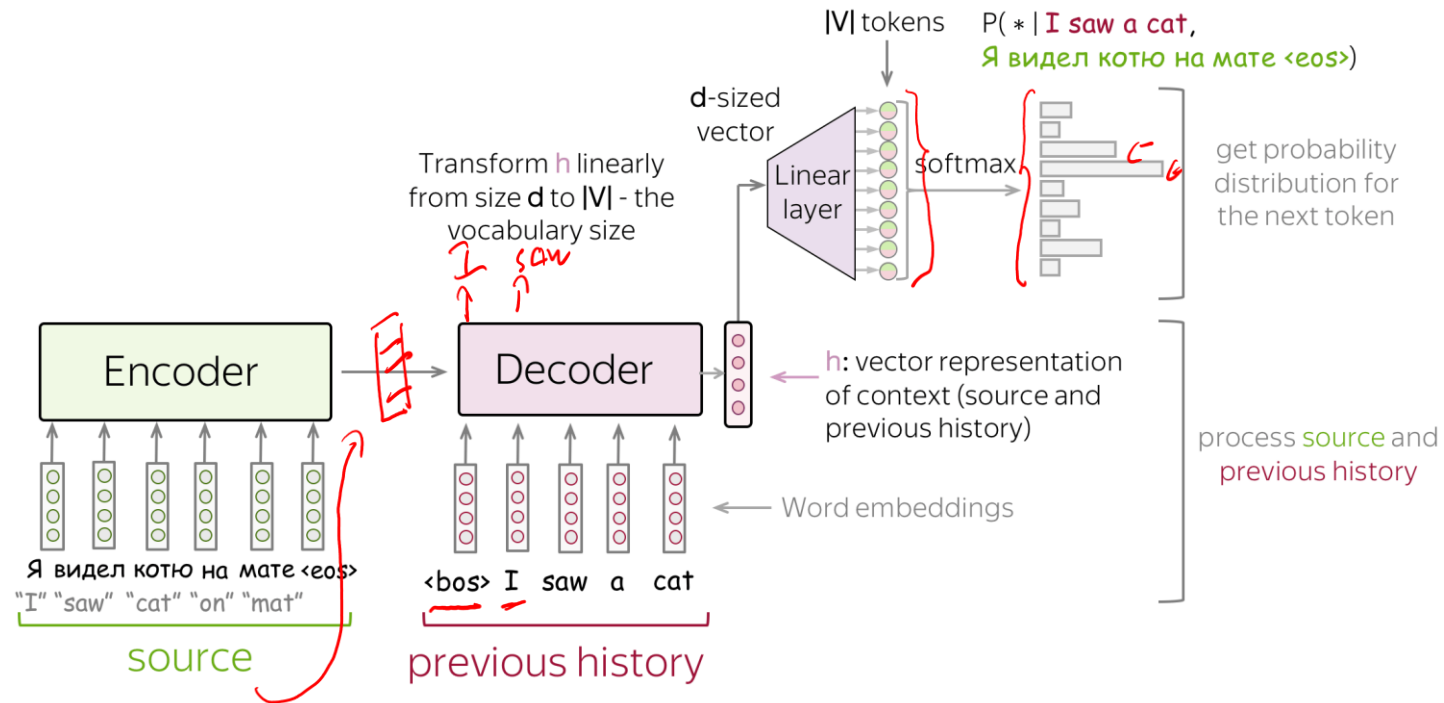
Encoder encapsulates a condition for a decoder Language Model

→ Language Models:
$$P(y_1, y_2, \dots, y_n) = \prod_{t=1}^n p(y_t | y_{<t})$$

→ Conditional
Language Models:
$$P(y_1, y_2, \dots, y_n, | x) = \prod_{t=1}^n p(y_t | y_{<t}, x)$$

condition on source x

A helicopter view on the Encoder-Decoder architecture



Model Loss is a well-known Cross-Entropy

Source sequence:

Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"

Target sequence:

I saw a cat on a mat <eos>

← one training example

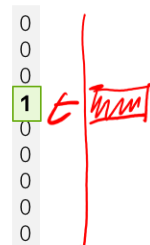
← one step for this example

previous tokens we want the model to predict this

Model prediction: $p(* | \text{I saw a, Я ... <eos>})$



Target



Loss = $-\log(p(\text{cat})) \rightarrow \min$

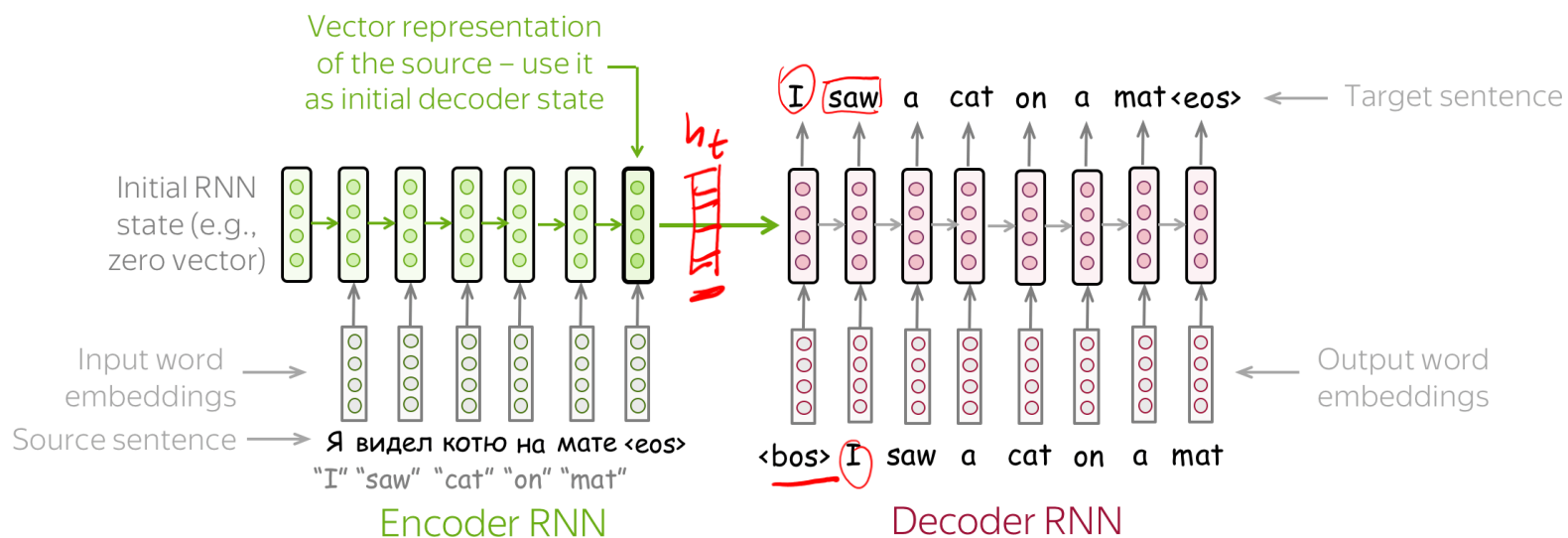


decrease

increase

decrease

A simple RNN is a valid Encoder-Decoder model

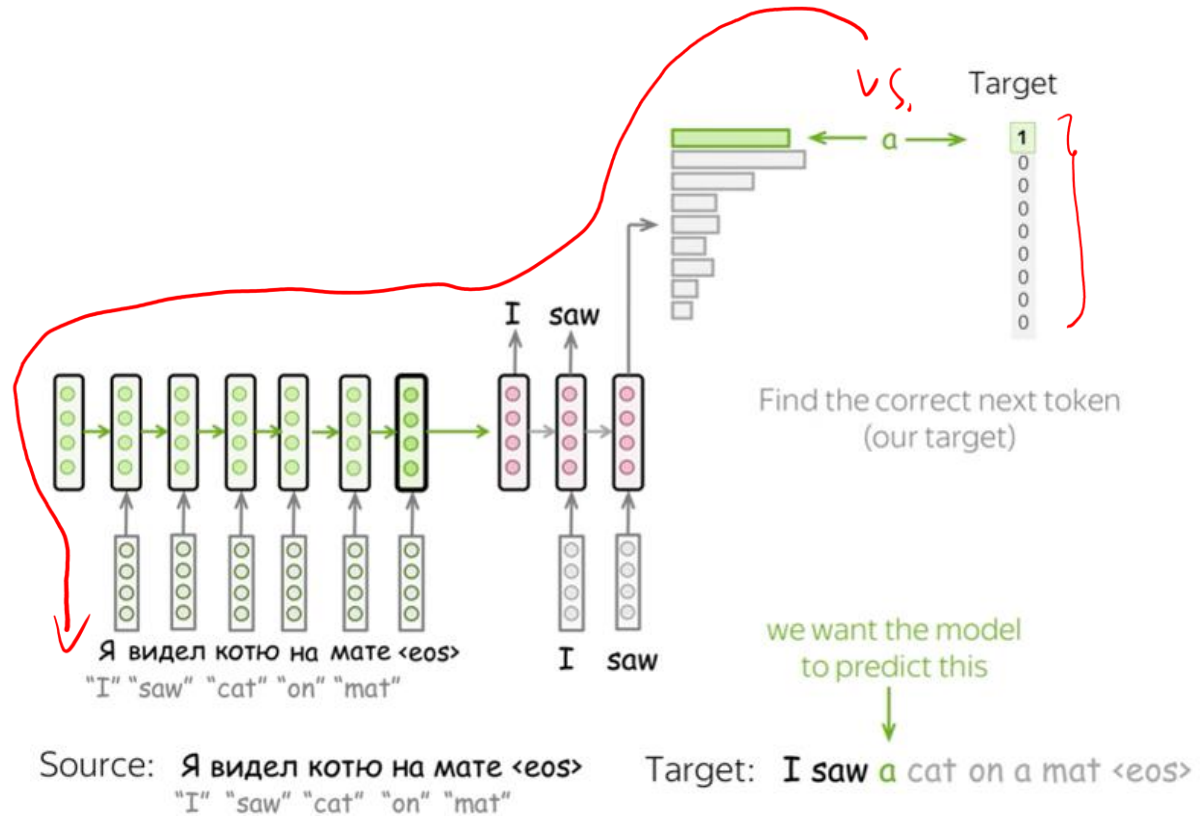


$$h_{t-1} \rightarrow \boxed{\text{RNN}} \rightarrow h_t$$

$$x \uparrow$$

$$(W_0 h_{t-1} + W_1 x) \rightarrow h_t$$

A simple RNN is a valid Encoder-Decoder model



Semantic space of text embeddings

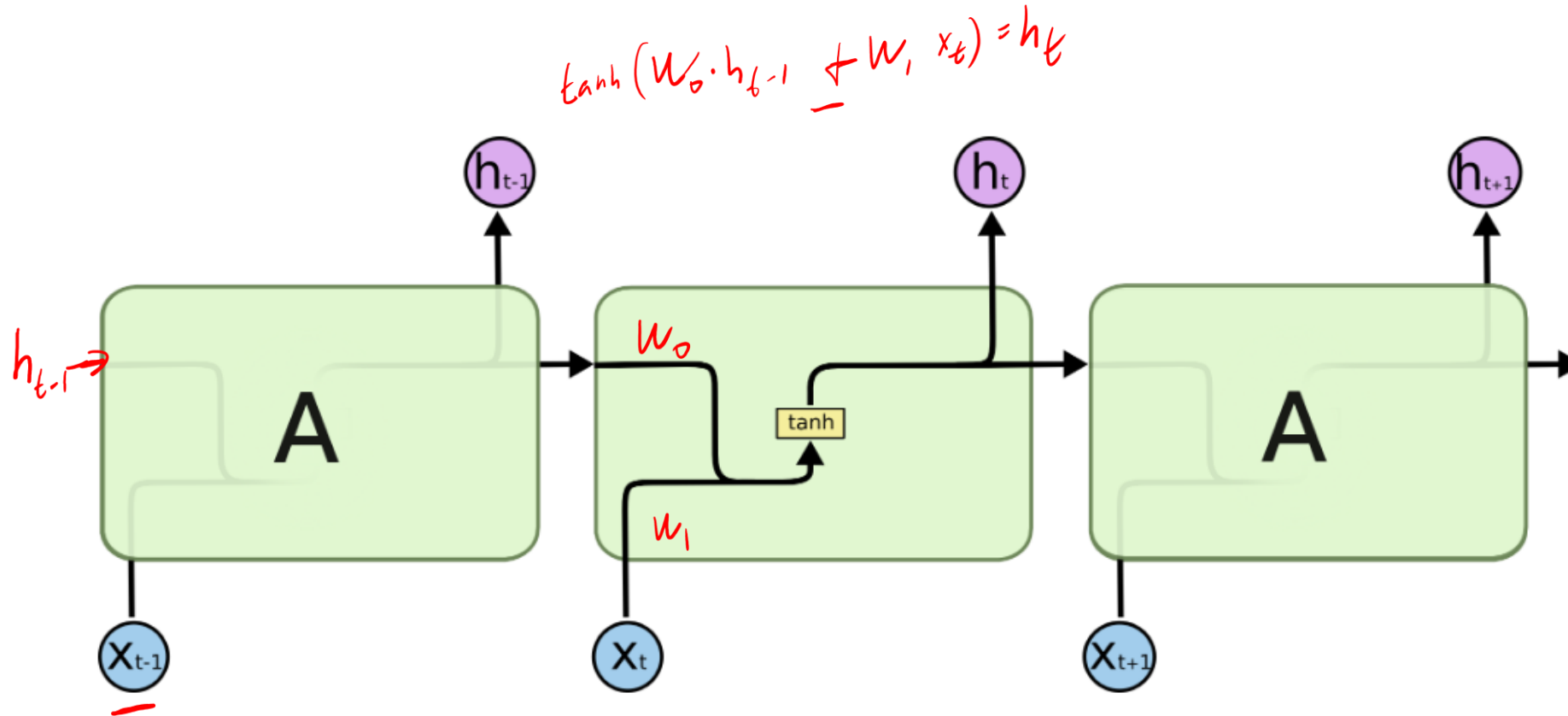




Table of Content

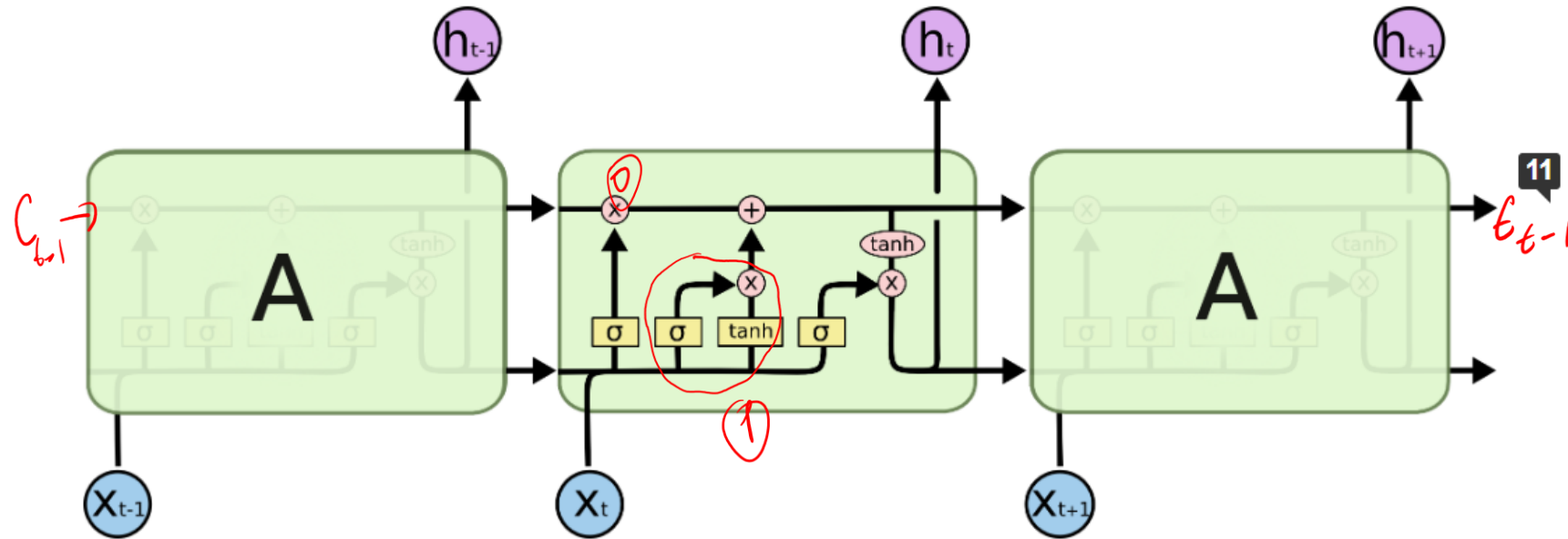
- Sequence to Sequence (seq2seq)
- **LSTM**
- Attention
- Practical tips

A vanilla RNN fails to account for long-term information



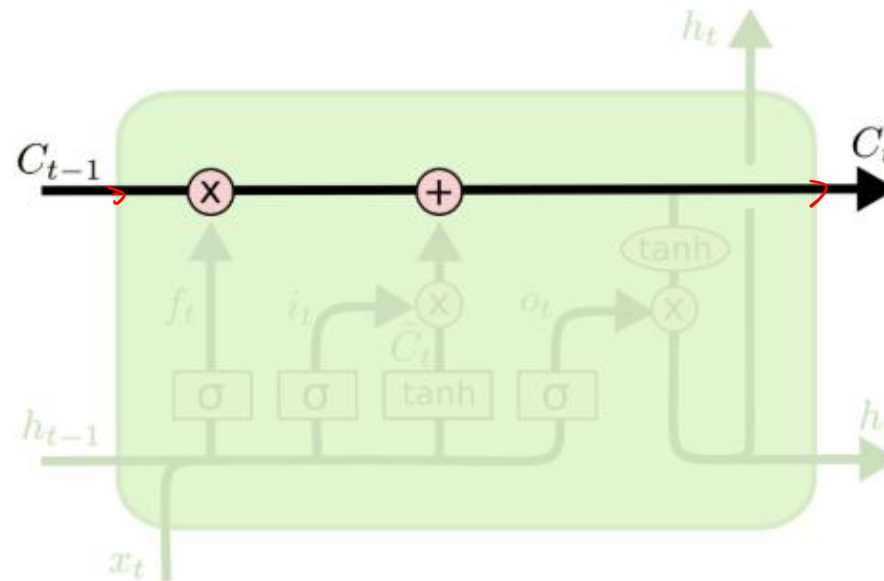
The repeating module in a standard RNN contains a single layer.

LSTM builds on an idea of RNN, introducing a separate channel for knowledge transmission

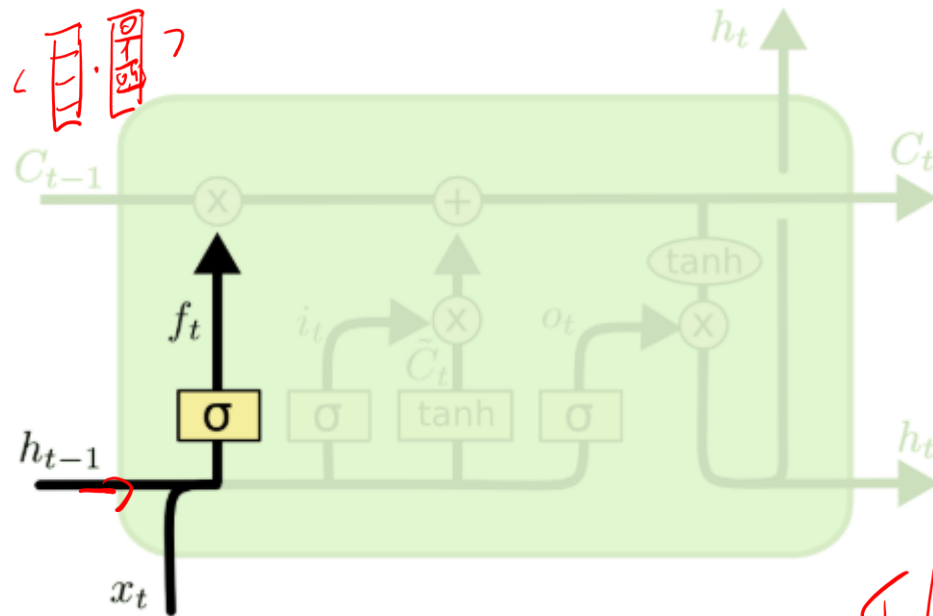


The repeating module in an LSTM contains four interacting layers.

The information flow is supported by the C channel



The idea is to control what new information is relevant to C

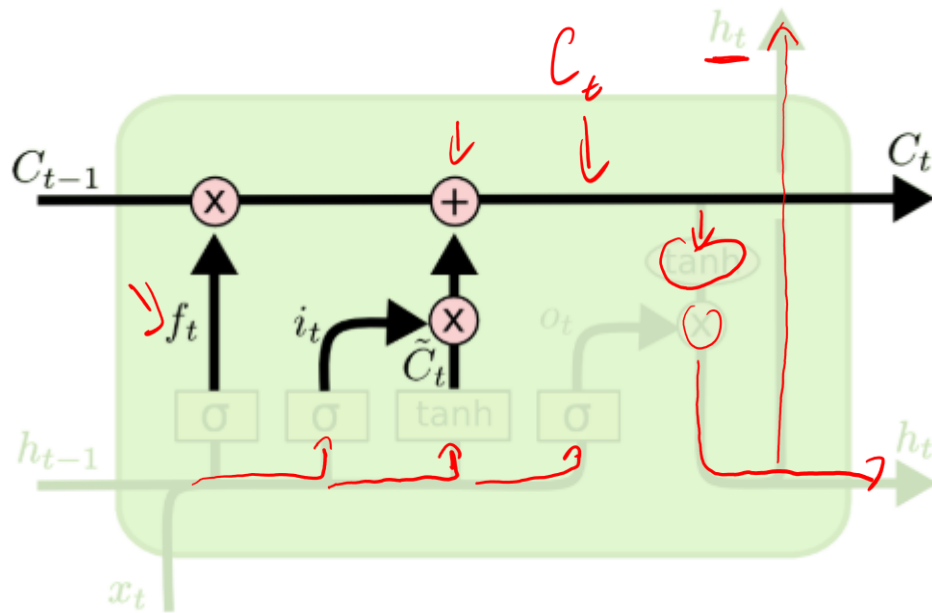


$$\int_1^2 \Delta (W_0 h_{t-1} + W_1 x_t + b_f)$$

$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

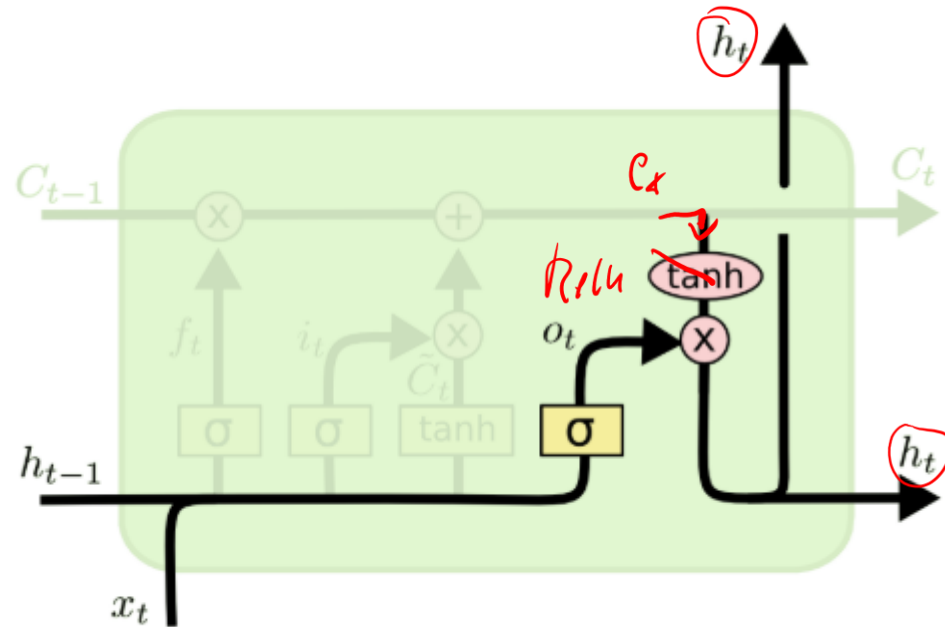
$$\sigma \left(\begin{bmatrix} W \end{bmatrix} \begin{bmatrix} \begin{bmatrix} \vdots \\ h_{t-1} \\ \vdots \end{bmatrix} \begin{bmatrix} \vdots \\ x_t \\ \vdots \end{bmatrix} \end{bmatrix} + \begin{bmatrix} b_f \\ \vdots \end{bmatrix} \right) = \sigma \left(\begin{bmatrix} \vdots \\ \vdots \end{bmatrix} \right)$$

The idea is to control what new information is relevant to C



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

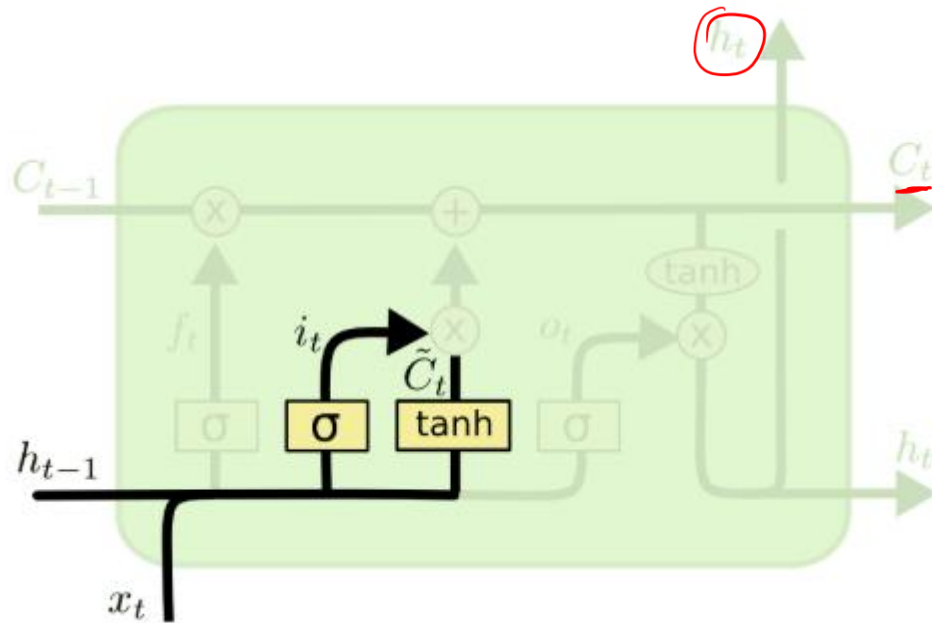
The idea is to control what new information is relevant to C



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

The idea is to control what new information is relevant to C



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

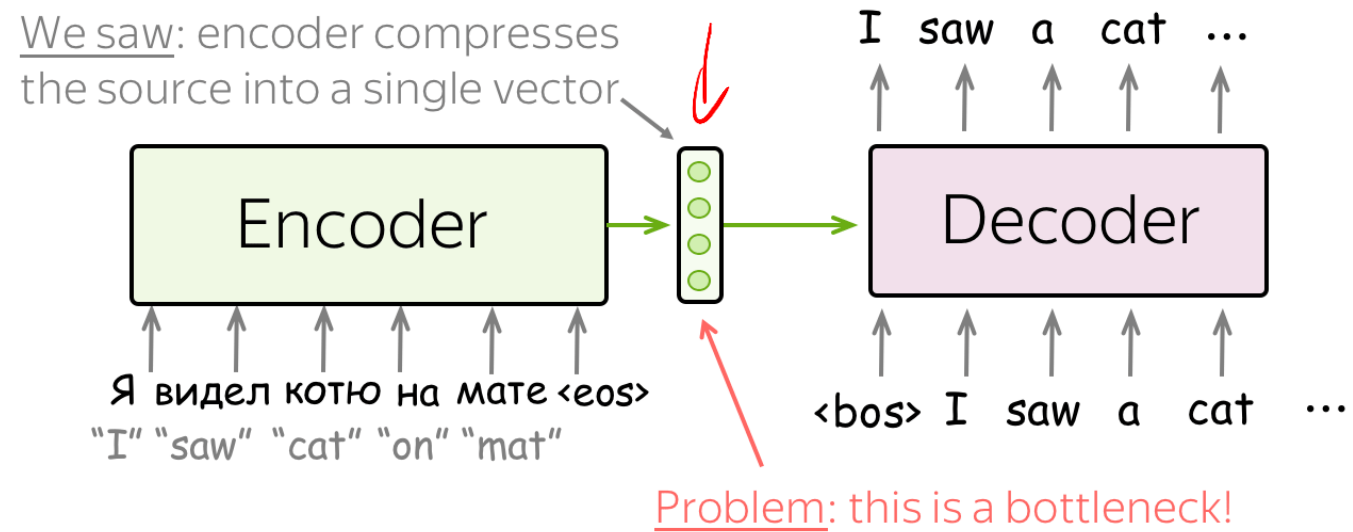
Handwritten red notes above the equations indicate the input vector $[h_{t-1}, x_t]$ and the output vector $[h_t, C_t]$.



Table of Content

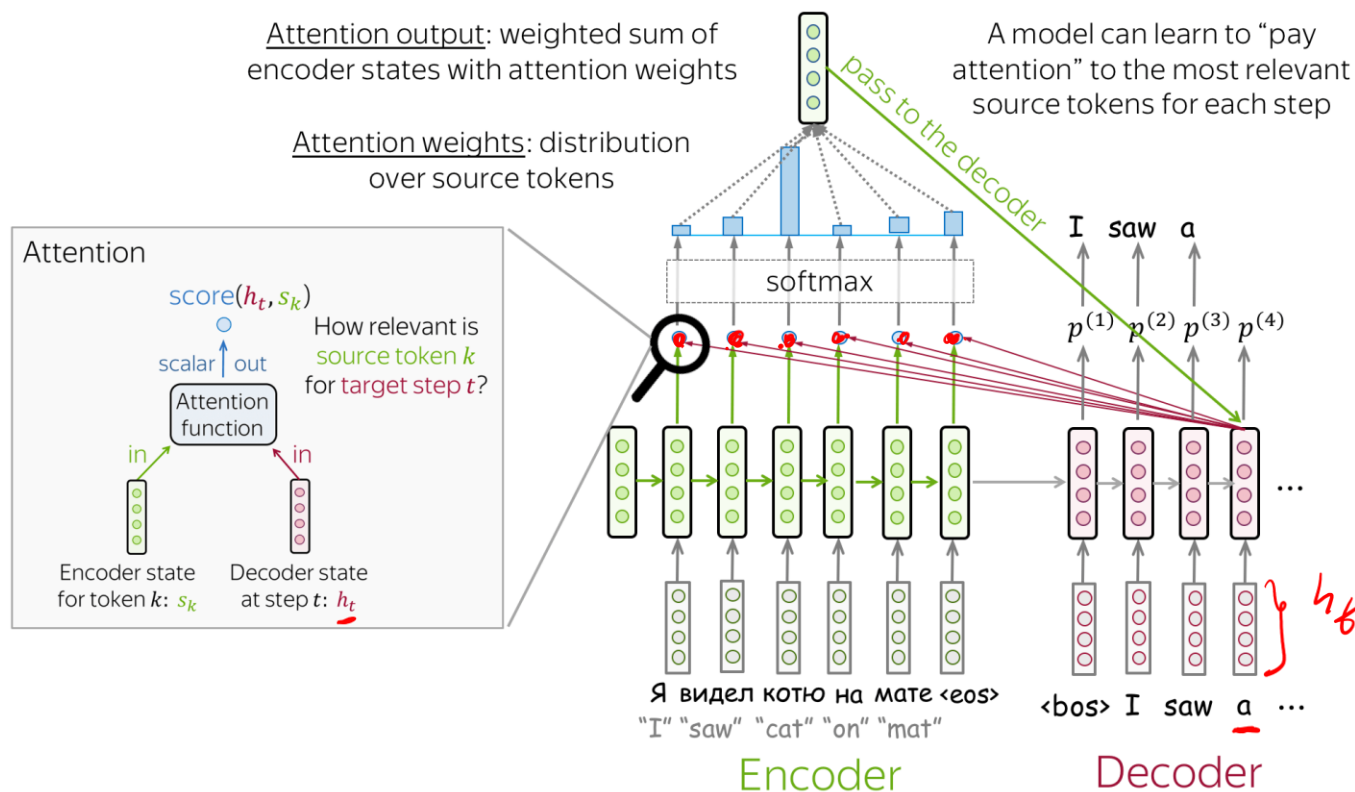
- Sequence to Sequence (seq2seq)
- LSTM
- **Attention**
- Practical tips

The final hidden state is a bottleneck

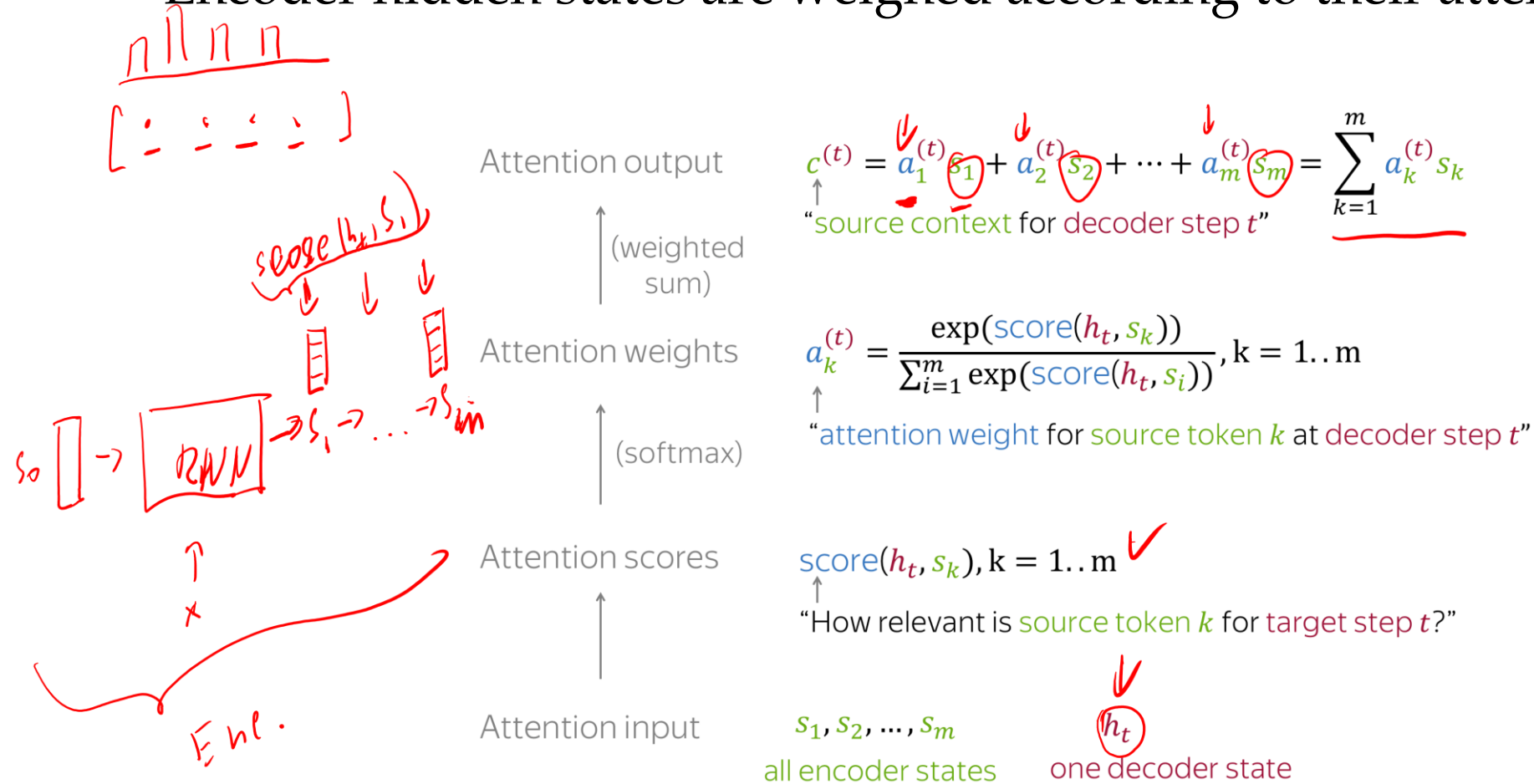


A simple Attention overview

$$\begin{bmatrix} - & h_1 & - \\ - & h_2 & - \\ & \vdots & \\ - & h_t & - \end{bmatrix} \begin{matrix} e \\ e \\ e \\ e \end{matrix} \begin{matrix} d \\ h_t \end{matrix}$$



Encoder hidden states are weighed according to their attention score



Encoder hidden states are weighed according to their attention score

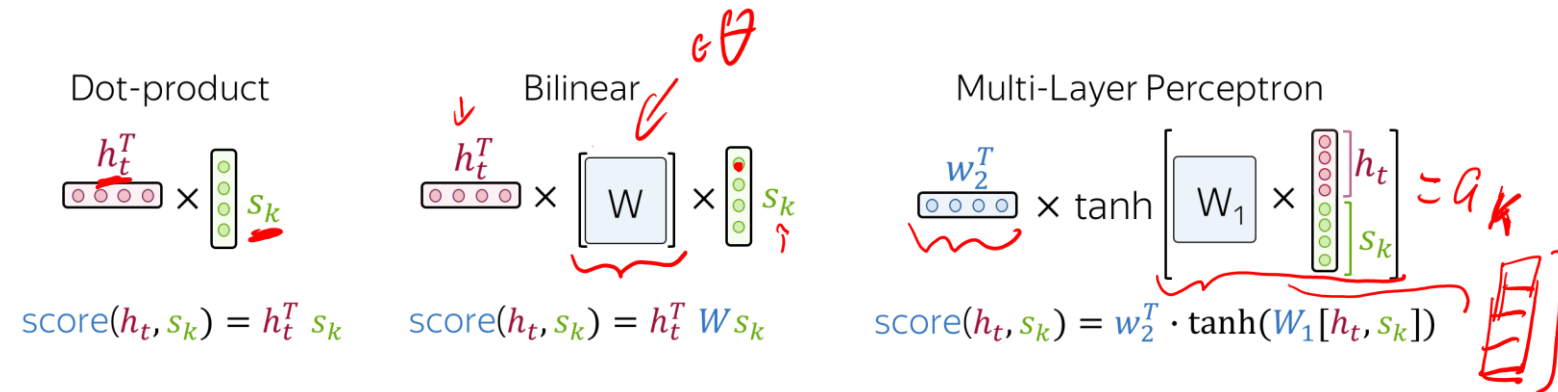


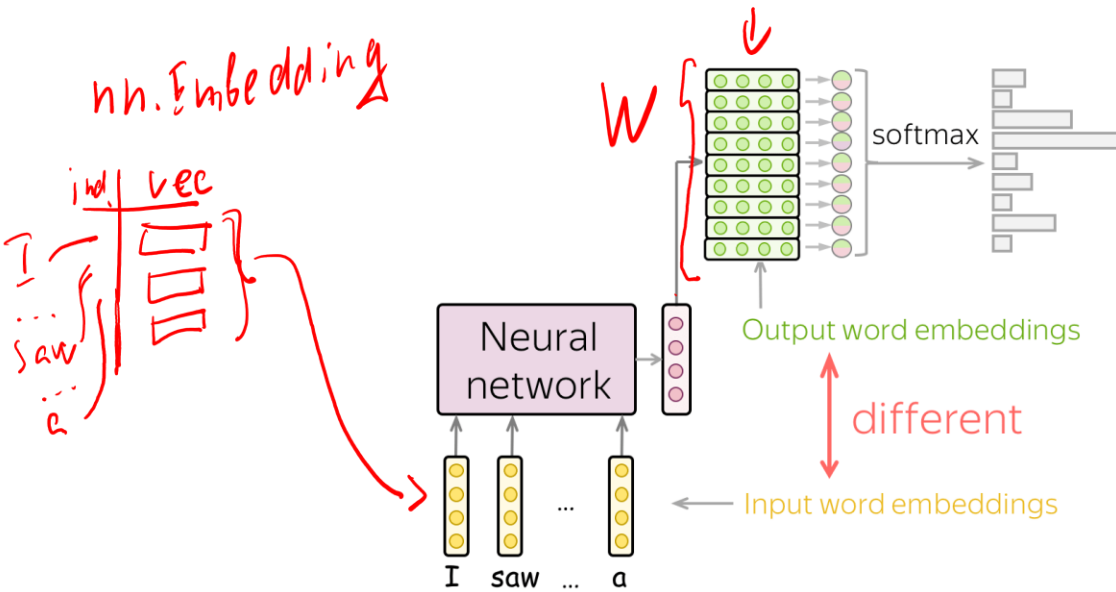


Table of Content

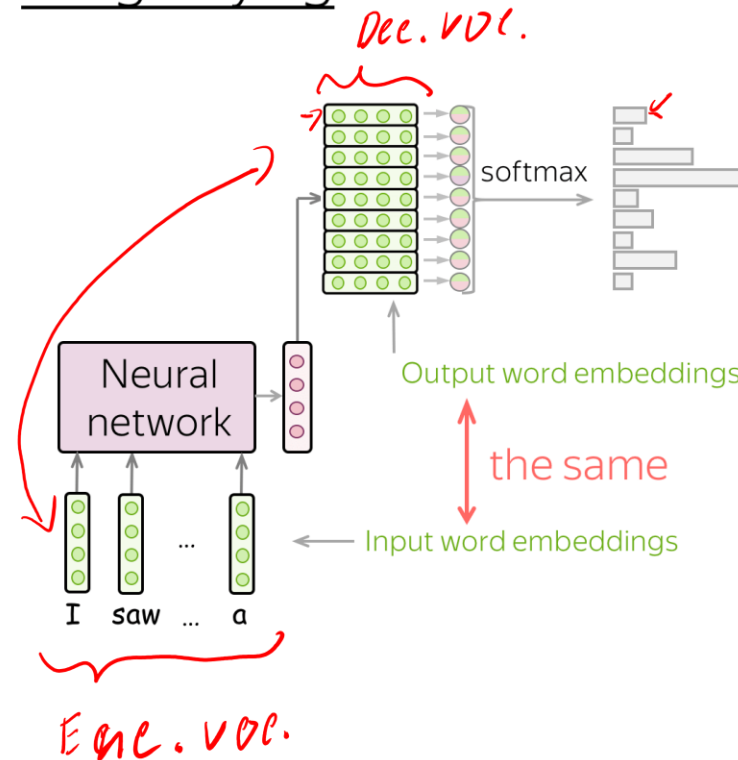
- Sequence to Sequence (seq2seq)
- Attention
- **Practical tips**

Weight tying is a way to significantly reduce the amount of parameters

Default (no weight tying)



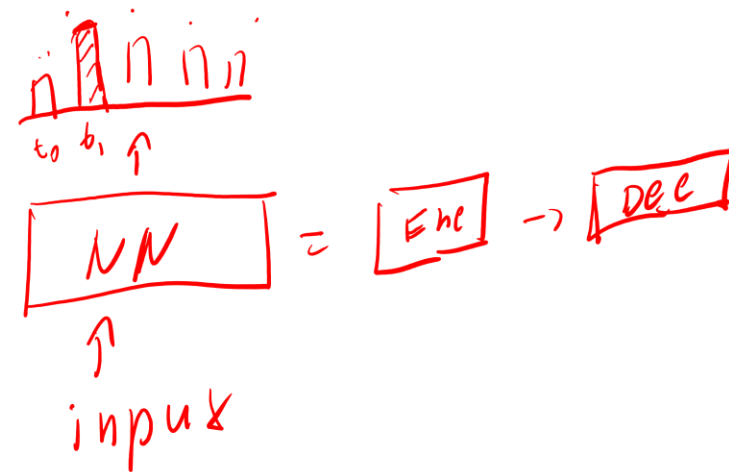
Weight tying



Finding the next token is not that trivial

$$\left\{ \begin{array}{l} p(t_j | t_i) \rightarrow \max \\ p(t_i | t_j) \rightarrow \max \end{array} \right. ; p(t_j | t_j) \rightarrow \max$$

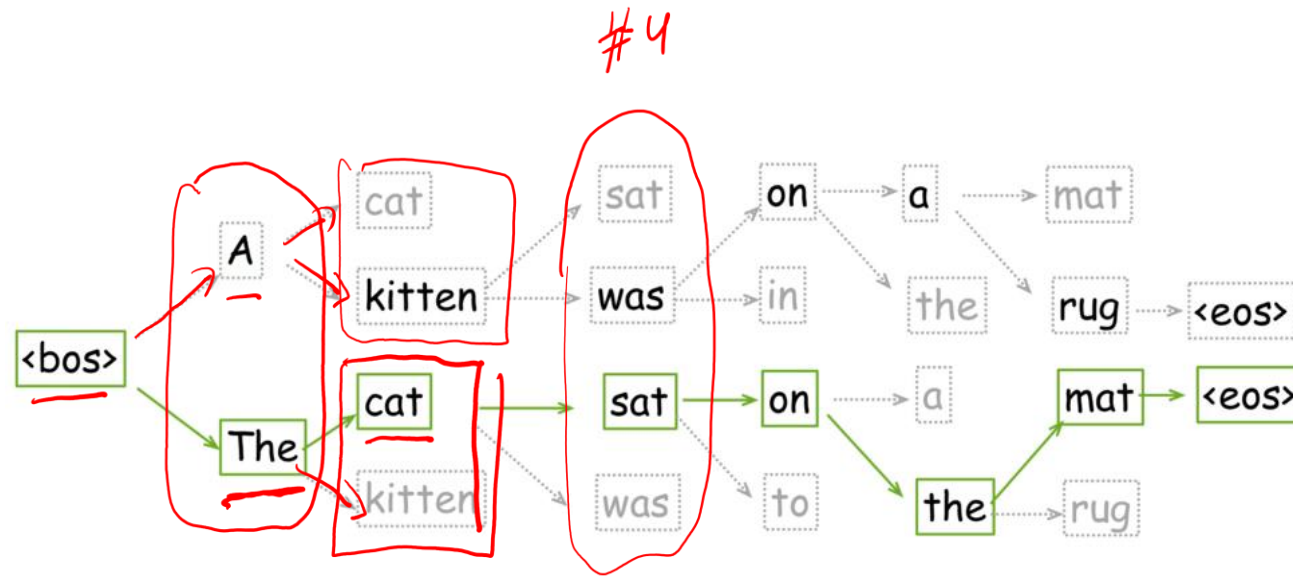
$$y' = \arg \max_y p(y|x) = \arg \max_y \prod_{t=1}^n p(y_t | y_{<t}, x) \quad \text{How to find the argmax?}$$



Greedy Decoding: At each step, pick the most probable token

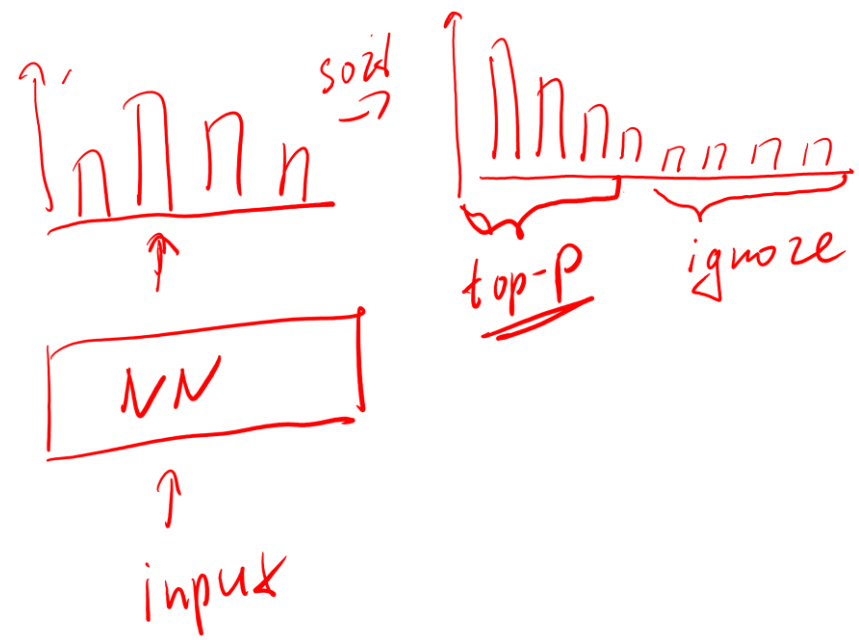
$$\arg \max_y \prod_{t=1}^n p(y_t | y_{<t}, x) \neq \prod_{t=1}^n \arg \max_{y_t} p(y_t | y_{<t}, x)$$

A beam search illustration



Pick the hypothesis with the highest probability

Top-p (nucleus) sampling



Temperature: the higher, the more chaotic the choice becomes

$$w_{next} \sim \frac{P(w_{next}|X)^{1/\tau}}{\sum_{\hat{w}} P(\hat{w}|X)^{1/\tau}}$$

